



COLLEGE CODE : 9605

COLLEGE NAME : CAPE INSTITUTE OF TECHNOLOGY

DEPARTMENT : BE.CSE(AI&ML),3rd YEAR

STUDENT NM-ID : CA61C0B86E5B0948A6D461DDF28C4C6C

ROLL NO : 960523148023

DATE : 25-10-2025

Completed the project named as Phase - 5

TECHNOLOGY PROJECT NAME : IBM -FE SINGLE PAGE APPLICATION

SUBMITTED BY,

NAME : VASANTH P

MOBILE NO : 9361107979

IBM -FE SINGLE PAGE APPLICATION



Final Demo Walkthrough – Explained



Setup

- **Open index.html directly in the browser**

Since this is a pure HTML/CSS/JS SPA, no server is needed. Just double-click index.html or drag it into your browser.

- **Ensure style.css, app.js, and sw.js are in the same directory**

These files handle styling, logic, and offline support. Keeping them together ensures everything loads correctly.

- **Navigation:** Show how clicking buttons changes views using window.location.hash (SPA routing).

- **Task or Feature Flow:** Demonstrate adding, editing, and deleting data (e.g., tasks or students) using forms.
- **Persistence:** Refresh the page to show that data is saved in localStorage.
- **Dark Mode:** Toggle the theme and reload to show it persists.
- **Offline Mode:** Use DevTools to simulate offline access and show the app still works (if using a service worker).
- **Responsive Design:** Resize the browser or use mobile view to show layout adapts.
- **Language Switching (optional):** Show how the app updates text dynamically when switching languages.
-  **Goal:** Prove that your SPA is interactive, persistent, responsive, and user-friendly—all without reloading the page.

Walkthrough Steps

1. Homepage

- **What to show:** A welcome message and navigation buttons like “Home” and “Tasks.”
- **Purpose:** Demonstrates SPA structure—content changes without reloading the page.

2. Routing

- **How it works:** Clicking “Tasks” updates `window.location.hash` (e.g., `#tasks`).
- **What to explain:** The app listens for `hashchange` events and renders the correct view dynamically.
- **Why it matters:** This simulates page navigation in a single-page app without using frameworks.

3. Task Creation

- **What to show:** A form to enter a task name.

- **What happens:** On submit, the task is saved to localStorage and displayed immediately.
- **Why it matters:** Demonstrates persistent data storage without a backend.

4. 🌙 Dark Mode

- **What to show:** Click the moon/sun toggle button.
- **What happens:** CSS classes switch between light and dark themes.
- **Persistence:** The theme choice is saved in localStorage and restored on reload.
- **Why it matters:** Enhances user experience and accessibility.

5. 📱 Offline Mode

- **How to simulate:** Open DevTools → Network tab → check “Offline.”
- **What to show:** Reload the app while offline—thanks to sw.js, it still works.

- **Why it matters:** Demonstrates Progressive Web App behavior using Service Workers.

6. Language Switching (optional)

- **What to show:** A dropdown to switch between English and Tamil.
- **What happens:** Text content updates dynamically using a translation object.
- **Why it matters:** Supports international users and improves accessibility.

7. Responsive Layout

- **How to test:** Resize the browser window or use DevTools → Device Toolbar.
- **What to show:** Layout adapts for mobile—buttons stack, text resizes, etc.
- **Why it matters:** Ensures usability across devices and screen sizes.

Project Report: Student Management System

1. Title

Student Management System – A Digital Platform for Academic Records

2. Abstract

This project is a web-based Student Management System designed to streamline the management of student data, including personal details, academic performance, attendance, and administrative records. It provides a user-friendly interface for administrators, teachers, and students to interact with the system efficiently.

3. Objectives

- Digitize student records and reduce manual paperwork.

- Enable quick access to student profiles, grades, and attendance.
- Provide role-based access for admin, faculty, and students.
- Ensure data integrity, security, and scalability.

4. Scope

- Manage student registration and profiles.
- Record and view academic scores.
- Track attendance and generate reports.
- Role-based dashboards for admin, teacher, and student.
- Export data for analysis or reporting.

5. Technologies Used

HTML

CSS

JavaScript

6. System Architecture

- **Frontend SPA:** Handles routing, UI rendering, and user interactions.
- **Data Layer:** Uses localStorage or mock JSON files for persistence.
- **Routing:** Hash-based navigation for SPA behavior.
- **Modular Components:** Reusable UI blocks for forms, tables, and dashboards.

7. Features

-   Student Registration & Profile Management
-  Academic Score Entry & Viewing
-  Attendance Tracking
-  Role-Based Access (Admin, Teacher, Student)
-  Report Generation (PDF/CSV optional)

- 🌙 Dark Mode Toggle
- 🌐 Multi-language Support (optional)

9. Challenges & Solutions

window.location.hash

localStorage

10. Future Enhancements

- Integrate backend with database (e.g., MongoDB).
- Add biometric attendance or QR code scanning.
- Enable email/SMS notifications.
- Add analytics dashboard for performance trends.

11. Conclusion

The Student Management System successfully demonstrates how modern web technologies can simplify academic administration. It offers a scalable,

user-friendly solution for managing student data and can be extended into a full-fledged institutional platform.

1. getTasks()

- **Purpose:** Retrieves all saved tasks.
- **How it works:** Reads the "tasks" key from localStorage and parses it into an array.
- **Example:**

```
const tasks =  
JSON.parse(localStorage.getItem("tasks") || "[]");
```

2. addTask(text)

- **Purpose:** Adds a new task with the given text.
- **How it works:** Creates a task object, appends it to the existing array, and updates localStorage.
- **Example:**

```
const tasks = getTasks();  
tasks.push({ id: Date.now(), text });
```

```
localStorage.setItem("tasks", JSON.stringify(tasks));
```

3. deleteTask(id)

- **Purpose:** Removes a task by its unique ID.
- **How it works:** Filters out the task with the matching ID and saves the updated list.

- **Example:**

```
const tasks = getTasks().filter(task => task.id !== id);  
localStorage.setItem("tasks", JSON.stringify(tasks));
```

4. toggleTheme()

- **Purpose:** Switches between light and dark mode.
- **How it works:** Toggles a CSS class on and stores the preference in localStorage.

- **Example:**

```
document.body.classList.toggle("dark");  
localStorage.setItem("theme",  
document.body.classList.contains("dark") ? "dark" :  
"light");
```

Project link:

Click here for project demo:

<https://vasanth-27072005.github.io/Single-page-application/>

Screenshots:



Student Management System

[Home](#) [Add Student](#) [About](#)



Student List

No students added yet. Click "Add Student" to get started!

© 2025 Student Management System | Built with ❤️ using HTML, CSS & JS





Student Management System

[Home](#)

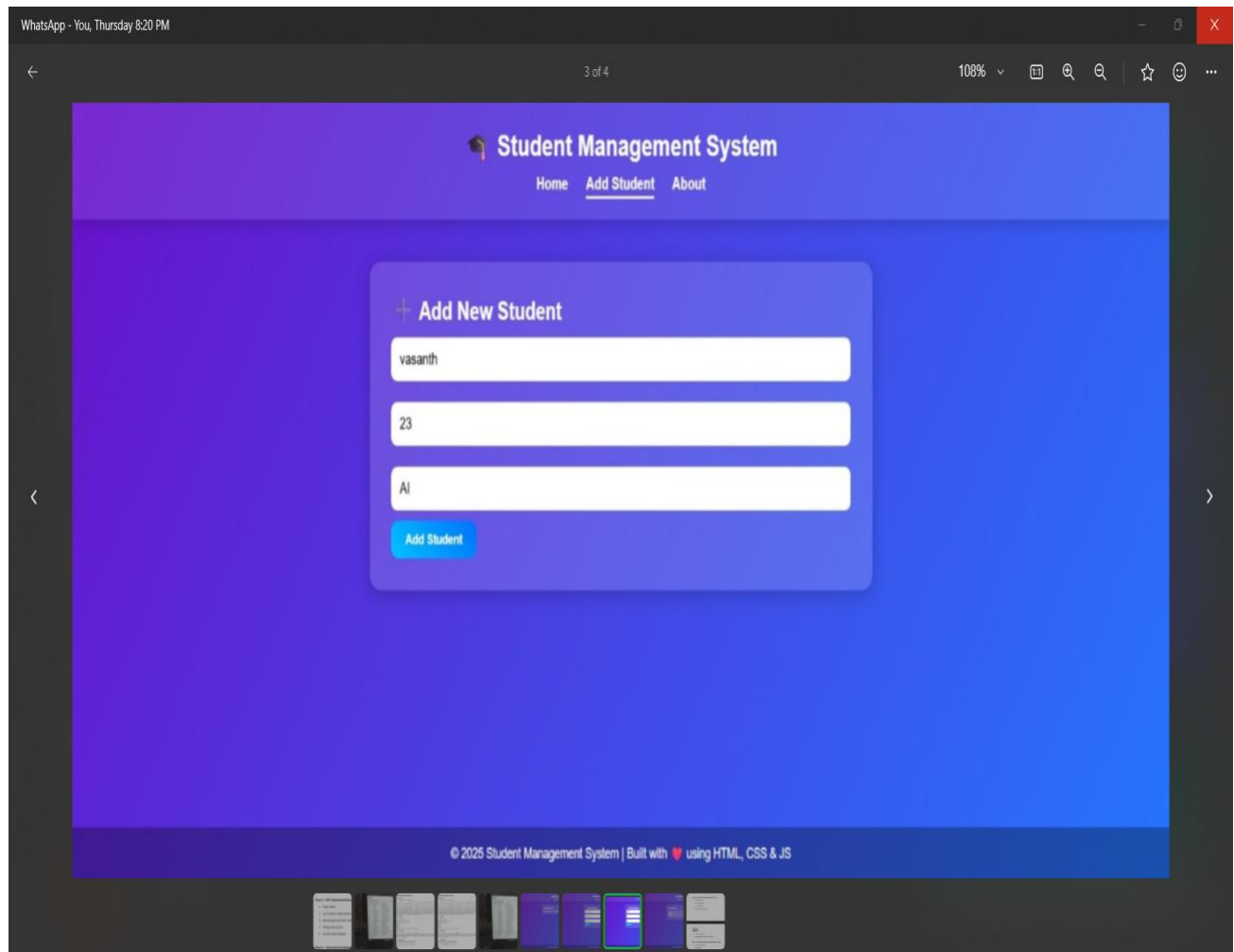
[Add Student](#)

[About](#)

+ Add New Student

Add Student





GitHub README & Setup Guide

README Sections

- **Project Title:** Student management system

- **Description:** A frontend-only SPA for managing tasks with modern UX features.
- **Live Demo:** (If hosted on GitHub Pages or Netlify)
- **Features:**
 - Task creation and persistence
 - Dark mode toggle
 - Offline access
 - Multi-language support
 - Responsive design
- **Tech Stack:** HTML, CSS, JavaScript
- **Setup Instructions:**
 - `git clone https://github.com/your-username/taskflow-lite.git`
 - `cd taskflow-lite`
 - open index.html in browser

GitHub_Setup_(1)[1].pdf

←

🔍

C:\Users\VASANTH\AppData\Local\Microsoft\Windows\NetCache\IE\KN8R4A1S\GitHub_Setup_(1)[1].pdf

☆

🔖

👤

⋮

🌈

☰

🔍

🖨

📄

🔗

⚙

1 of 10

🔍

🖨

📄

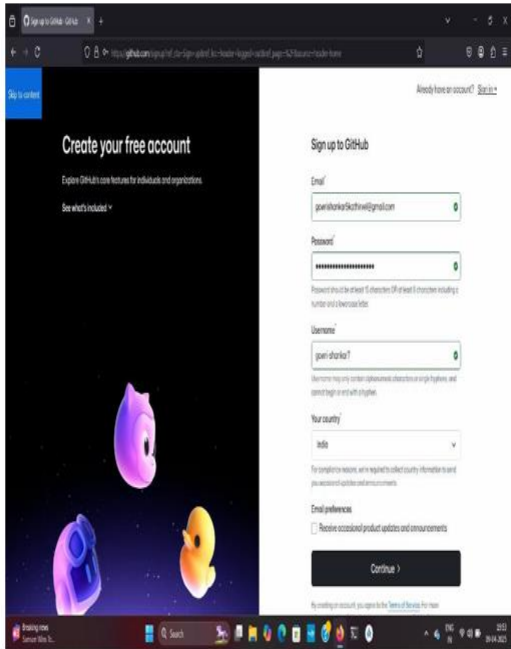
🔗

⚙

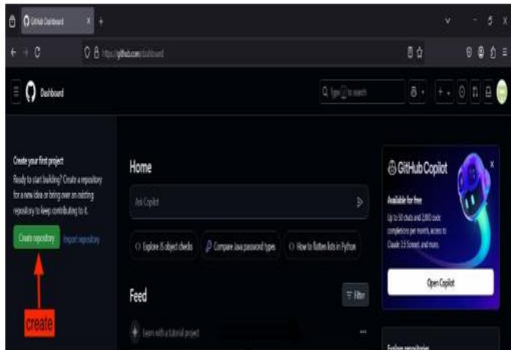
Edit with Acrobat

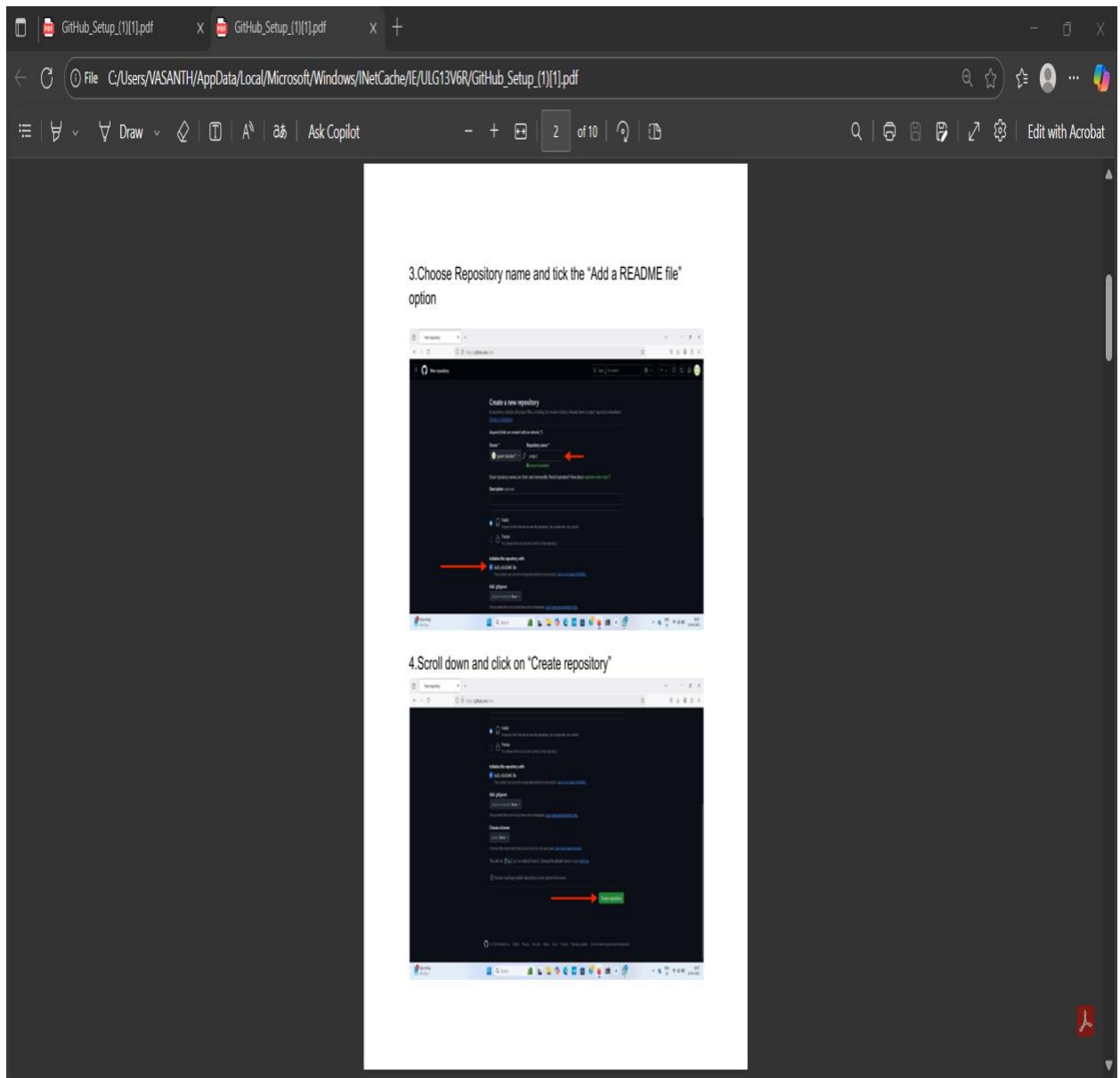
GitHub Setup

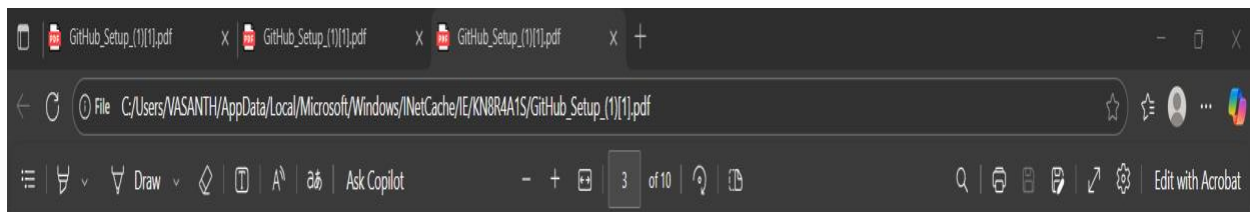
1. Visit [GitHub Signup](#) and create an account



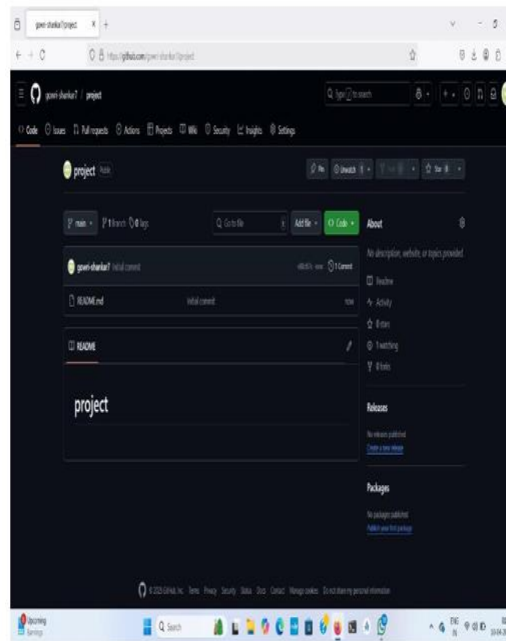
2. Click on "Create repository"



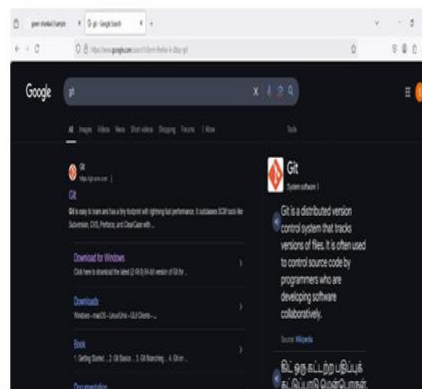


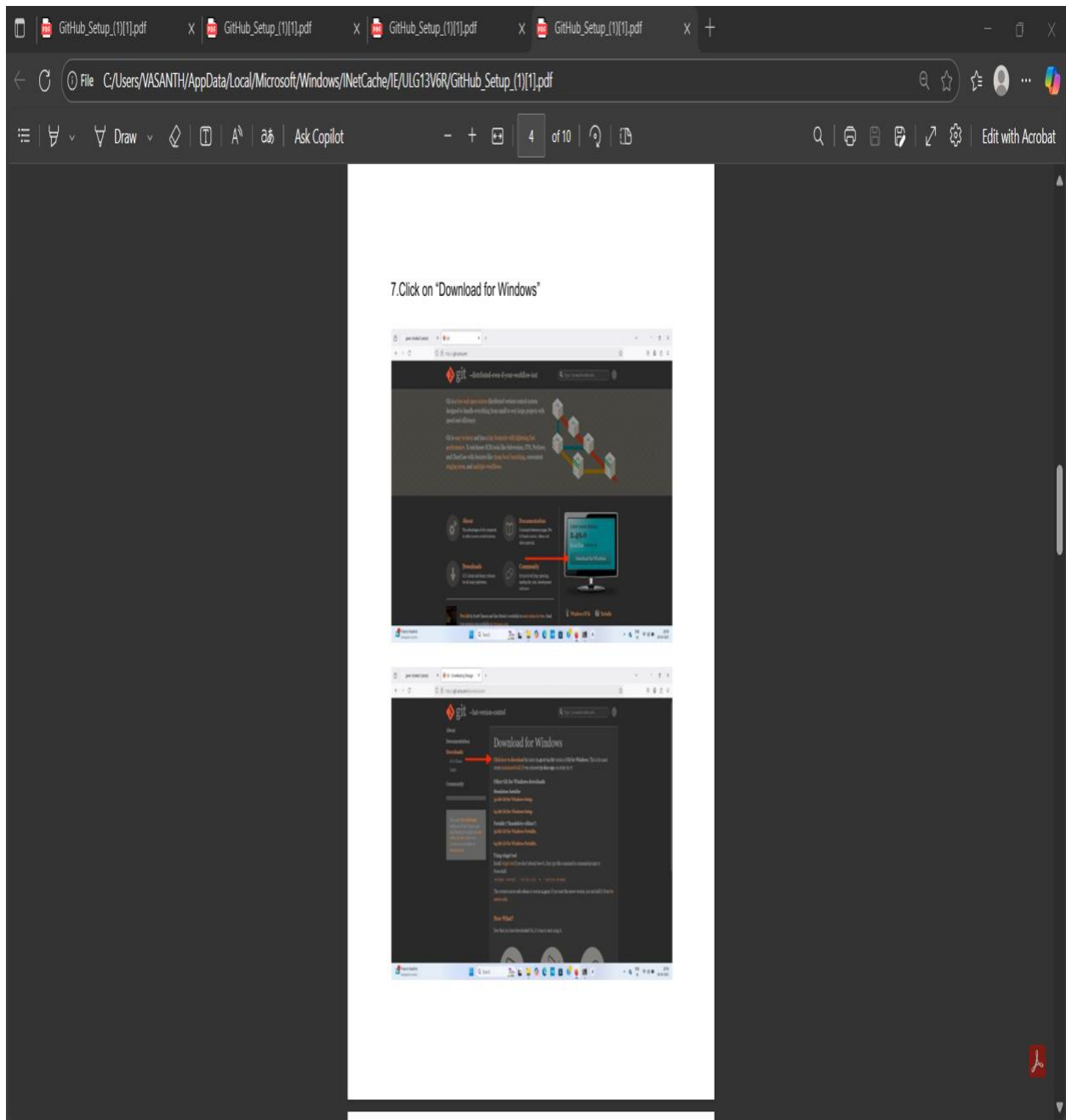


5.This the repository page

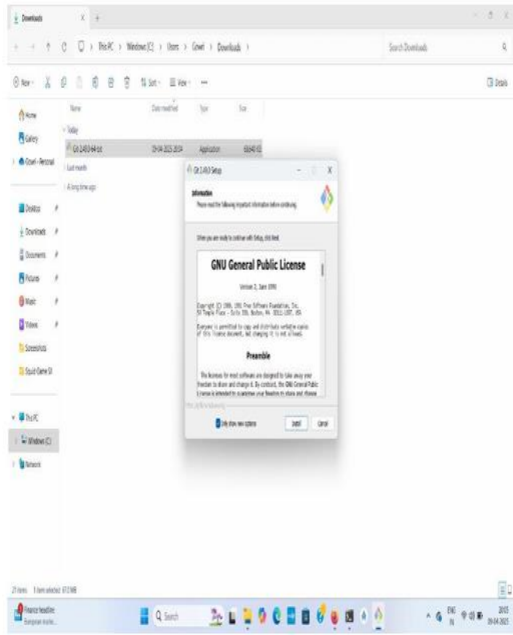


6.search "git"



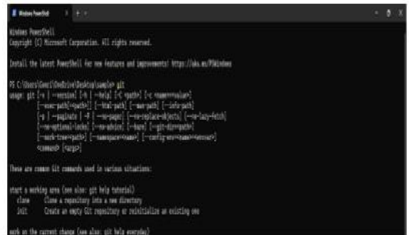


8. Install Git from Downloads



9. After installation, Verify using the following command in terminal

```
git
```



Code:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>Student Management System</title>

  <link rel="stylesheet" href="spa.css">

</head>

<body>

  <header>

    <h1>🎓 Student Management System</h1>

    <nav>

      <a href="#home" class="active">Home</a>

      <a href="#add">Add Student</a>

      <a href="#about">About</a>

    </nav>

  </header>

  <main id="content"></main>
```

```
<footer>
```

```
    <p>© 2025 Student Management System | Built with 
using HTML, CSS & JS</p>
```

```
</footer>
```

```
<script src="spa.js"></script>
```

```
</body>
```

```
</html>
```

```
/* ----- Global Styles ----- */
```

```
* {
```

```
    margin: 0;
```

```
    padding: 0;
```

```
    box-sizing: border-box;
```

```
    transition: all 0.3s ease;
```

```
}
```

```
body {
```



```
font-family: 'Poppins', sans-serif;
background: linear-gradient(120deg, #6a11cb, #2575fc);
color: white;
min-height: 100vh;
display: flex;
flex-direction: column;
}
```

```
/* ----- Header & Navigation ----- */
```

```
header {
  background: rgba(255, 255, 255, 0.1);
  backdrop-filter: blur(12px);
  padding: 20px;
  text-align: center;
  box-shadow: 0 3px 15px rgba(0,0,0,0.2);
}
```

```
h1 {
  font-size: 28px;
```

```
margin-bottom: 10px;  
}
```

```
nav {  
  display: flex;  
  justify-content: center;  
  gap: 25px;  
}
```

```
nav a {  
  color: #fff;  
  text-decoration: none;  
  font-weight: 600;  
  position: relative;  
  padding-bottom: 5px;  
}
```

```
nav a::after {  
  content: ";
```

```
position: absolute;
bottom: 0;
left: 0;
width: 0%;
height: 2px;
background: #fff;
transition: width 0.3s;
}
```

```
nav a:hover::after, nav a.active::after {
width: 100%;
}
```

```
/* ----- Main Content ----- */
main {
flex: 1;
padding: 40px;
display: flex;
justify-content: center;
```

```
align-items: flex-start;
}
```

```
.container {
  background: rgba(255,255,255,0.15);
  padding: 30px;
  border-radius: 15px;
  width: 90%;
  max-width: 700px;
  backdrop-filter: blur(10px);
  box-shadow: 0 4px 20px rgba(0,0,0,0.2);
  animation: fadeIn 0.6s ease;
}
```

```
/* ----- Buttons & Inputs ----- */
```

```
input {
  width: 100%;
  padding: 12px;
  margin: 10px 0;
```

```
border: none;
border-radius: 8px;
font-size: 15px;
}
```

```
button {
  background: linear-gradient(45deg, #00c6ff, #0072ff);
  border: none;
  color: white;
  padding: 10px 20px;
  font-weight: bold;
  border-radius: 10px;
  cursor: pointer;
}
```

```
button:hover {
  transform: scale(1.05);
  background: linear-gradient(45deg, #0072ff, #00c6ff);
}
```

```
/* ----- Student Cards ----- */
```

```
.student-card {  
    background: rgba(255,255,255,0.2);  
    border-left: 5px solid #00c6ff;  
    padding: 15px;  
    border-radius: 10px;  
    margin-bottom: 12px;  
    display: flex;  
    justify-content: space-between;  
    align-items: center;  
}
```

```
.student-card button {  
    background: #ff5e5e;  
    padding: 6px 10px;  
    border-radius: 5px;  
}
```

```
.student-card button:hover {  
    background: #ff2e2e;  
}
```

```
/* ----- Footer ----- */  
footer {  
    text-align: center;  
    background: rgba(0, 0, 0, 0.3);  
    padding: 15px;  
    font-size: 14px;  
}
```

```
/* ----- Animations ----- */  
@keyframes fadeIn {  
    from {opacity: 0; transform: translateY(10px);}  
    to {opacity: 1; transform: translateY(0);}  
}
```

```
/* ----- Responsive ----- */
```

```
@media (max-width: 600px) {
```

```
  nav{
```

```
    flex-direction: column;
```

```
  }
```

```
.container{
```

```
  width: 100%;
```

```
  padding: 20px;
```

```
}
```

```
}
```

```
const content = document.getElementById("content");
```

```
const navLinks = document.querySelectorAll("nav a");
```

```
window.addEventListener("hashchange", router);
```

```
window.addEventListener("load", router);
```

```
let students = JSON.parse(localStorage.getItem("students")) || [];
```

```
// Navigation Highlight
```

```
function setActiveLink(hash) {
```

```
  navLinks.forEach(link => link.classList.remove("active"));
```



```
document.querySelector(`a[href="${hash}"]`)?.classList.add("active");
}
```

// Router

```
function router() {
  const hash = window.location.hash || "#home";
  setActiveLink(hash);
  if (hash === "#home") showHome();
  else if (hash === "#add") showAddForm();
  else if (hash === "#about") showAbout();
}
```

// Home Page

```
function showHome() {
  let html = `<div class="container">
    <h2> Student List</h2><hr style="margin: 10px 0; border-color: #fff;">
    `;
  if (students.length === 0) {
    html += `<p>No students added yet. Click "Add Student" to get started!</p>`;
  }
}
```

```
} else {  
  students.forEach((s, i) => {  
    html += `  
      <div class="student-card">  
        <div>  
          <strong>${s.name}</strong><br>  
          Roll No: ${s.roll}<br>  
          Dept: ${s.dept}  
        </div>  
        <button onclick="deleteStudent(${i})">Delete</button>  
      </div>`;   
    });  
  }  
  html += `</div>`;   
  content.innerHTML = html;  
}
```

// Add Student Page

```
function showAddForm() {  
  content.innerHTML = `  
    <div class="container">
```

```

<h2>✚ Add New Student</h2>
<input id="name" placeholder="Enter Student Name">
<input id="roll" placeholder="Enter Roll Number">
<input id="dept" placeholder="Enter Department">
<button onclick="addStudent()">Add Student</button>
</div>`;
}

function addStudent() {
  const name = document.getElementById("name").value.trim();
  const roll = document.getElementById("roll").value.trim();
  const dept = document.getElementById("dept").value.trim();

  if (name && roll && dept) {
    students.push({ name, roll, dept });
    localStorage.setItem("students", JSON.stringify(students));
    alert("✅ Student added successfully!");
    window.location.hash = "#home";
  } else {
    alert("⚠️ Please fill all fields!");
  }
}

```

```
}
```

```
// Delete Student
```

```
function deleteStudent(index) {
```

```
  if (confirm("Are you sure you want to delete this student?")) {
```

```
    students.splice(index, 1);
```

```
    localStorage.setItem("students", JSON.stringify(students));
```

```
    showHome();
```

```
  }
```

```
}
```

```
// About Page
```

```
function showAbout() {
```

```
  content.innerHTML = `
```

```
    <div class="container">
```

```
      <h2>💡 About This Project</h2>
```

```
      <p>This modern Single Page Application is built using only  
<strong>HTML, CSS, and JavaScript</strong>.</p>
```

```
      <p>It allows users to add, view, and delete student records  
without any page reloads.</p>
```

<p>Data is saved in your browser using
localStorage so it persists even after
refreshing.</p>

<p style="margin-top:10px; color:#ffeb3b;">Designed with  by
You!</p>

</div>`;

}